SİSTEM MİMARİSİ (MONOLITH vs MICROSERVICES & KATMANLI MİMARİ)

"Bu sistem fiziksel olarak nasıl duracak, bileşenler birbiriyle nasıl konuşacak?"

---

1. SİSTEM MİMARİSİ NEDİR? (Neden Bu Kadar Kritik?)

Sistem Mimarisi, bir yazılım sisteminin **temel yapısını**, bu yapıyı oluşturan bileşenleri, bu bileşenlerin birbirleriyle ve dış dünyayla olan ilişkilerini, ayrıca bu yapıyı yöneten **tasarım prensiplerini** ve kararlarını tanımlayan soyut çerçevedir.

> **Kurumsal Gerçek:** Mimari kararları, proje başlangıcında alınır ve değiştirmek, gökdelenin temelini değiştirmek gibidir. Çok maliyetlidir ve çok zaman alır. Amazon, Netflix, Uber gibi devler, başlangıçta monolith iken, ölçeklenme sorunu yaşadıkları için mikroservislere geçmek zorunda kaldılar. Bu geçiş yıllar sürdü.

---

2. KATMANLI MİMARİ (LAYERED ARCHITECTURE) – KLASİK VE SAĞLAM

Katmanlı mimari, en yaygın ve en anlaşılır mimari modelidir. Bir sistemin, sorumluluklarına göre **dikey katmanlara** (layer) ayrılmasıdır. Her katman, yalnızca bir alt katmanla iletişim kurar (üstten alta doğru). Bu, bağımlılığı azaltır ve bakımı kolaylaştırır.

Klasik 3 Katmanlı Mimarı (3-Tier Architecture)

Katman	Gösterim	Sorumluluk	Gerçek Dünya Örneği
Sunum Katmanı (Presentation Layer)	En Üst	Kullanıcı arayüzü (UI), kullanıcı girdisini alır ve sonuçları gösterir. Kullanıcı deneyiminden sorumludur.	Web sayfası (React), Mobil uygulama (Swift/Android), API Gateway (Swagger UI)
İş Mantığı Katmanı (Business Logic Layer)	Orta	Sistemin "beyni"dir. Kuralları, hesaplamaları, iş akışlarını, karar mekanizmalarını içerir.	Kredi skoru hesaplama, sipariş onaylama, fiyatlandırma, stok kontrolü.
Veri Erişim Katmanı (Data Access Layer)	Alt	Veritabanıyla iletişim kurar. CRUD (Create, Read, Update, Delete) işlemlerini gerçekleştirir. SQL sorguları, ORM (Entity Framework, Hibernate) bu katmandadır.	Müşteri bilgilerini veritabanından getir, siparişi kaydet.

Neden Katmanlı Mimarı?

Artılar	Eksiler / Zorluklar
Modülerlik: Her katman bağımsız geliştirilebilir.	**Performans kaybı:** Her istek, katmanlar arasında dolaşır (Presentation → Business → Data → Business → Presentation). Ekstra ağ ve işlem yükü.
Bakım Kolaylığı: Sunum katmanını değiştirirseniz (ör. Web'den Mobil'e) iş mantığı ve veri katmanı dokunulmadan kalır.	**Teknolojik uyum zorluğu:** Tüm katmanlar aynı dil/teknoloji ile yazılmak zorunda değildir ama genelde öyledir.
Test Edilebilirlik: Her katman, ayrı ayrı birim testine tabi tutulabilir.	**Dağıtım zorluğu:** Ufak bir iş mantığı değişikliği için tüm uygulamanın yeniden dağıtılması gerekir (Monolith eğilimi).
Güvenlik: Sunum katmanı, iş mantığına doğrudan erişemez. Tüm trafik kontrollüdür.	**Ölçeklenebilirlik sınırı:** Tüm uygulama tek bir sunucuda çalışıyorsa, sadece bir katmanı (ör. veritabanı) ölçeklendiremezsiniz.

Katmanlı Mimaride "Katman" ve "Katmanlar Arası İletişim" Kuralları

Kural	Açıklama
Yukarıdan Aşağıya Bağımlılık	Sunum katmanı, iş mantığına bağlıdır. İş mantığı, veri erişimine bağlıdır. Tersine bağımlılık (Veri erişiminin iş mantığına bağlanması) mimari koku (architectural smell) olarak kabul edilir.
Katmanlar Arası Geçiş	Sunum katmanı, doğrudan veri erişim katmanına erişemez. Tüm işlemler mutlaka iş mantığından geçer. Bu, iş kurallarının atlanmasını önler.
DTO (Data Transfer Object) Kullanımı	Katmanlar arasında veri taşımak için özel sınıflar (DTO) kullanılır. Veritabanı entity'si, doğrudan sunum katmanına gönderilmez. Bu, güvenlik ve soyutlama sağlar.

3. MONOLITH MİMARİ (TEK PARÇA) – KLASİK BAŞLANGIÇ

Monolith, tüm uygulamanın (sunum, iş mantığı, veri erişimi, arka plan işleri, mesaj kuyrukları) **tek bir dağıtılabilir birim** (tek bir JAR/WAR dosyası, tek bir .exe) halinde paketlenip, **tek bir sunucuda veya tek bir işlemde** çalıştırıldığı mimari modelidir.

Monolith'in Özellikleri

Özellik	Açıklama	Örnek (Global E-Ticaret)
Tek Kod Tabanı	Tüm geliştiriciler aynı koda commit yapar.	Sipariş, Ürün, Kullanıcı, Ödeme, Raporlama hepsi aynı projede.
Tek Dağıtım	Yeni bir özellik (ör. kupon sistemi) eklendiğinde, tüm uygulama yeniden derlenir, test edilir ve dağıtılır.	Haftada 1 kez deploy (büyük sürümler).

Özellik	Açıklama	Örnek (Global E-Ticaret)
Tek Ölçeklendirme	Uygulama yoğunluğu arttığında, tüm uygulama kopyalanır (yeni sunucu).	3 sunucuya yük dengeleme (load balancer) ile dağıtılır.

Monolith'in Artıları ve Eksileri (Kurumsal Bakış)

Artılar	Eksiler / Zorluklar
Basit Geliştirme: Yeni başlayanlar için kolaydır. Tek bir proje, tek bir dil.	**Devasa Kod Tabanı:** Yüzbinlerce satır kod, yeni gelenler için öğrenmesi imkansız.
Kolay Debug: Tüm kod tek bir yerde olduğu için hata ayıklamak (debug) basittir.	**Kırılganlık (Fragile):** Tek bir modül (ör. raporlama) çökerse, tüm uygulama çöker.
Kolay Dağıtım: Tek bir dosyayı kopyalamak yeterlidir.	**Ölçeklenebilirlik Sorunu:** Sadece "sipariş" yoğunluğu arttığında, tüm uygulamayı (raporlama, ürün) ölçeklendirmek zorundasınız. Kaynak israfı.
Teknolojik Homojenlik: Herkes aynı teknolojiyi kullanır.	**Teknoloji Bağımlılığı:** Yeni bir teknoloji (ör. NoSQL) denemek için tüm uygulamayı değiştirmek gerekir.
Başlangıç İçin Ucuz: Küçük ekipler ve MVP (Minimum Viable Product) için idealdir.	**Geliştirme Hızı Düşer:** 50+ geliştirici aynı kodda çalışırsa, merge conflict (çakışma) kabusa döner.

> **Kurumsal Gerçek:** Amazon, 2000'li yıllarda monolith bir kitap satış sitesiydi. Ancak büyüdükçe, tüm sistemin 10 dakika deploy olması, "Prime Now" gibi hızlı teslimat özelliklerini imkansız kıldı. Bu yüzden mikroservislere geçtiler.

---

4. MİKROSERVİS MİMARİSİ (MSA) – DEVLERİN SAVAŞI

Mikroservis Mimarisi, uygulamanın, **kendi başına çalışabilen, bağımsız dağıtılabilen, belirli bir iş yeteneğine (business capability) odaklanmış küçük servisler** halinde parçalanmasıdır. Her servis, kendi veritabanına, kendi kod tabanına ve kendi dağıtım döngüsüne sahiptir. Servisler, hafif protokollerle (HTTP/REST, gRPC, mesaj kuyrukları) birbirleriyle haberleşir.

Mikroservis Mimarisi Örneği (Global E-Ticaret)

Servis	Sorumluluk	Kendi Veritabanı	Teknoloji
Kullanıcı Servisi	Kullanıcı kaydı, giriş, profil yönetimi.	PostgreSQL	Java Spring Boot
Ürün Servisi	Ürün CRUD, stok, kategori.	MongoDB (NoSQL)	Node.js Express
Sipariş Servisi	Sipariş alma, durum güncelleme.	PostgreSQL	Python Django
Ödeme Servisi	Kredi kartı doğrulama, ödeme işleme.	Oracle (mevcut legacy)	Java JEE

Servis	Sorumluluk	Kendi Veritabanı	Teknoloji
Kargo Servisi	Kargo etiketi, takip numarası, teslimat.	MySQL	Go
Bildirim Servisi	SMS, E-posta, Push bildirim gönderme.	Redis (ön bellek)	.NET Core

Mikroservisler Arası İletişim (Synchronous vs Asynchronous)

İletişim Tipi	Protokol	Açıklama	Örnek (Sipariş Servisi → Ödeme Servisi)
Senkron (Synchronous)	REST API / HTTP	Servis A, Servis B'ye istek atar, yanıtı bekler. Basittir ama bağımlılık yaratır.	Sipariş Servisi, Ödeme Servisi'ne REST ile "Ödeme yap" der. Yanıt gelene kadar bekler. (Hata durumunda tüm işlem başarısız olur.)
Asenkron (Asynchronous)	Message Queue (RabbitMQ, Kafka)	Servis A, bir mesaj kuyruğuna istek atar ve beklemez. Servis B, kuyruktan alır ve işler. Gevşek bağlıdır (loose coupling).	Sipariş Servisi, "SiparişOnaylandı" mesajını Kafka'ya atar. Ödeme Servisi bunu dinler ve işler. Ödeme gecikse bile sipariş düşmez.

Mikroservislerin Artıları ve Eksileri (Kurumsal Bakış)

Artılar	Eksiler / Zorluklar
Bağımsız Dağıtım: Ödeme servisini güncellerken, Sipariş servisi çalışmaya devam eder. Haftada 100+ deploy.	**Karmaşıklık (Dağıtık Sistem):** Servisler arası iletişim, hata yönetimi, zaman aşımı (timeout) gibi konular çok zordur.
Teknoloji Özgürlüğü: Her servis, işe en uygun teknolojiyle yazılabilir. (Ödeme için Java, Raporlama için Python).	**Veri Tutarlılığı (Distributed Transactions):** 2 farklı servis 2 farklı veritabanını güncelliyorsa, "Hepsini yap ya da hiçbirini yap" (ACID) mümkün değildir. Saga pattern kullanılır.
Esnek Ölçeklenme: Sadece yoğunluğu artan servis (ör. Ödeme) ölçeklendirilir. Diğer servisler aynı kalır.	**Operasyonel Yük:** Her servis ayrı ayrı izlenmeli (monitoring), loglanmalı, deploy edilmelidir. Devops ekibi gerektirir.
Küçük Takımlar: 2 pizza kuralı (bir takım 2 pizzayı paylaşacak kadar küçük olmalı). Her takım bir servisten sorumludur.	**Ağ Gecikmesi:** Servisler ağ üzerinden konuştuğu için, monolith'e göre daha yavaştır. (Extra network hop).
Yenilikçilik: Yeni bir servis (ör. yapay zeka destekli öneri) mevcut sistemi etkilemeden deneyebilirsiniz.	**Test Zorluğu:** Tüm servisleri ayağa kaldırarak entegrasyon testi yapmak çok zor ve maliyetlidir.

5. MONOLITH vs MİKROSERVİS – KARAR MATRİSİ (NE ZAMAN HANGİSİ?)

Bu kararı verirken aşağıdaki kriterleri değerlendirirsiniz. Hiçbiri "her zaman doğru" değildir.

Kriter	Monolith Önerilir	Mikroservis Önerilir
Ekip Büyüklüğü	Küçük ekip (5-10 kişi)	Büyük ekip (50+ kişi)
Proje Tipi	MVP, Startup, Prototip	Kurumsal, Ölçeklenebilir, Kritik sistem
Değişim Hızı	Yavaş (ayda 1 deploy)	Çok hızlı (günde 10+ deploy)
Teknoloji Çeşitliliği	Tek dil/teknoloji	Her servis farklı teknoloji kullanabilir
Veri Tutarlılığı	ACID, kesin tutarlılık gereklidir	Nihai tutarlılık (Eventual Consistency) kabul edilebilir.
Operasyonel Altyapı	Basit VM veya tek sunucu	Kubernetes, Docker, Servis Mesh, izleme sistemleri (Prometheus, Grafana) mevcuttur.
Bütçe	Düşük (tek sunucu)	Yüksek (30+ sunucu, container orchestration)

> **Kurumsal Strateji:** Netflix, Amazon, Uber gibi devler **önce monolith ile başlar, sonra stratejik olarak mikroservislere ayrışır (Strangler Fig Pattern)**. Yani, eski sistemin yanına yeni mikroservis eklenir, trafik yavaş yavaş yeni servislere yönlendirilir. Bu, daha az riskli ve kontrollü bir geçiştir.

6. KATMANLI MİMARİ İLE MİKROSERVİS BİRLİKTE Mİ ÇALIŞIR?

Evet! Aslında her mikroservis, kendi içinde **katmanlı mimari (3-Tier)** kullanır. Yani her mikroservis, Presentation (API katmanı), Business ve Data katmanlarına sahiptir. Fark, bu servislerin **ayrı işlemlerde** çalışmasıdır.

Örnek: Sipariş Mikroservisi, kendi içinde:

- **Presentation:** REST Controller (HTTP isteklerini alır)
- **Business:** Sipariş validasyonu, toplam fiyat hesaplama
- **Data:** Sipariş veritabanına (PostgreSQL) CRUD

Aynı zamanda Sipariş Servisi, Ödeme Servisi ile (senkron/asenkron) iletişim kurar.

7. SİSTEM ANALİSTİNİN MİMARİ KARARLARINDAKİ ROLÜ

Aşama	Analistin Görevi
Gereksinim Analizi	"Sistemin ölçeklenme ihtiyacı nedir? Kaç kullanıcı? Gelecekte hangi modüller eklenecek?" sorularını sorar.

Aşama	Analistin Görevi
Mimari Alternatifleri Sunma	Monolith ve Mikroservis'in artı/eksilerini, iş birimine (Product Owner, Yöneticiler) anlaşılır bir dille anlatır.
Veri Modeli ile Eşleştirme	ER diyagramındaki tabloların hangi servise ait olacağını belirler. (Örn: Kullanıcı tablosu = Kullanıcı Servisi).
İletişim Protokollerini Belirleme	Hangi servislerin senkron (REST) hangilerinin asenkron (Kafka) konuşacağına karar verirken iş ihtiyaçlarını temel alır. (Örn: Ödeme işlemi senkron olmalı, ancak raporlama asenkron olabilir).
Dokümantasyon	Mimari karar dokümanı (ADR - Architectural Decision Record) hazırlar. Neden monolith değil de mikroservis seçildiğini, riskleri ve alternatifleri yazar.

---

8. ÖZET

- **Mimari** = Sistemin iskeleti. Temel değişirse her şey çöker.
- **Katmanlı Mimari** = Sunum, İş Mantığı, Veri Erişimi. Her katman bir alt katmana bağımlıdır. (Temel yapı taşı).
- **Monolith** = Tek parça, tek dağıtım, tek ölçekleme. Başlangıç için idealdir. Ancak büyüdükçe bakım işkenceye döner.
- **Mikroservis** = Bağımsız servisler, bağımsız veritabanları, bağımsız dağıtım. Ölçeklenebilir ve esnektir, ama operasyonu zordur.
- **Strateji:** Önce monolith, büyüdükçe stratejik olarak mikroservislere ayrıştır (Strangler Pattern).

SORU

Senaryo: "Global Sigorta" şirketi, mevcut hasar tazminat sistemini (monolith, 20 yıllık, Cobol altyapısı) yenilemek istiyor. Yeni sistemde;

- Müşteri talepleri,
- Eksper atama,
- Hasar tutarı hesaplama,
- Ödeme onayı,
- Raporlama modülleri bulunacak. Şirketin 100+ geliştiricisi var, yılda 2 büyük sürüm çıkarıyor. Yeni sistemle, rakip firmalara göre daha hızlı yeni özellik (ör. anlık hasar fotoğraf yükleme) sunmak istiyor.

Soru:

1. Bu projede **Monolith** mi yoksa **Mikroservis** mi seçersiniz? Neden? (Ekip büyüklüğü, yenilik hızı, veri tutarlılığı ihtiyacı açısından değerlendirin).
2. Eğer Mikroservis seçerseniz, kaç servis önerirsiniz? Hangi servisler (hangileri ayrı veritabanına sahip olmalı?) olur?
3. Hasar tutarı hesaplama ile ödeme onayı arasındaki iletişim senkron mu yoksa asenkron mu olmalı? Gerekçesiyle açıklayın.

Global Sigorta: Hasar Tazminat Sistemi Modernizasyonu — Mimari Değerlendirme

Verilen senaryo; klasik bir **monolit → modüler/dağıtık mimari** dönüşüm senaryosu. Aşağıda bu karar için kritik analizleri ve önerilen yaklaşım verilmiştir.

1. Temel Sorun Tespiti

Bulgu	Anlamı
20 yıllık Cobol monolit	Değişiklik maliyeti yüksek, bilgi tek kişilerde (bus factor riski)
100+ geliştirici, yılda 2 sürüm	Monolit, bu ölçekte koordinasyon darboğazı; "2 sürüm" kısıtı doğrudan mimariden kaynaklanıyor
"Rakiplerden hızlı özellik" hedefi	Time-to-market ana KPI; mimari bu hedefe hizmet etmeli
5 net modül sınırı	Doğal bounded context adayları mevcut

Kritik içgörü: Yılda 2 sürüm, 100+ geliştiriciyle çalışmanın *sonucu*, sebebi değil. Monolitin release treni modeli, bağımsız dağıtımı imkânsız kılıyor. Hedef, **bağımsız deploy edilebilir modüller** olmalı.

2. Modül Ayrıştırma (Domain-Driven Design)

Mevcut modüller zaten iyi bir başlangıç noktası, ancak sınırlar yeniden çizilmeli:

Bounded Context	Sorumluluk	Not
Talep Yönetimi	Müşteri talepleri, dosya açma, belge/foto yükleme	"Anlık fotoğraf yükleme" buradan başlar → **ilk pilot modül için ideal**
Eksper Atama	Eksper havuzu, atama kuralları, iş yükü	Talep'ten ayrı; kendi yaşam döngüsü var
Hasar Hesaplama	Tutar hesaplama, poliçe limitleri, muafiyetler	**En karmaşık domain** — izole edilmeli, iyi test edilmeli
Ödeme Onayı	Onay akışları, limitler, ödeme tetikleme	Finansal kontrol; audit kritik
Raporlama	Operasyonel + yönetimsel raporlar	**Ayrı servis değil, ayrı veri ürünü** — event/CDC ile beslenen read-model

Önemli uyarı: Raporlama modülünü diğerlerinin içine gömmeyin. Raporlama sorguları transactional sistemleri yorar; event-driven beslenen ayrı bir read-model (CQRS yaklaşımı) hem performans hem bağımsızlık sağlar.

3. Önerilen Geçiş Stratejisi: "Strangler Fig" + Modüler Monolit Başlangıcı

Anti-pattern uyarısı: 100+ geliştiriciyle doğrudan "mikroservis"e atlamak yaygın bir hatadır. Dağıtık sistem karmaşıklığı (observability, veri tutarlılığı, ağ hataları) henüz olgunlaşmamış bir ekibe ağır gelir.

Önerilen yol:

1. **Faz 0 — Modüler Monolit (ilk 6-9 ay):** Yeni sistemi tek deploy birimi ama *kesin modül sınırlarıyla* (ayrı paketler, ayrı şemalar, modüller arası yalnızca API/event iletişim) başlatın. Bu, DDD sınırlarını düşük maliyetle doğrular.
2. **Faz 1 — Pilot modül:** "Talep Yönetimi + fotoğraf yükleme" ile başlayın. Hem iş değeri yüksek (rakip farkı) hem de diğer modüllerden en az bağımlı alan.
3. **Faz 2 — Strangler Fig ile Cobol'u kemirin:** Eski sistem önüne bir facade koyun; trafiği modül modül yeni sisteme kaydırın. Cobol'u "big bang" ile kapatmayın.
4. **Faz 3 — Seçici ayrıştırma:** Yalnızca ölçek/bağımsızlık ihtiyacı kanıtlanmış modülleri (ör. Hasar Hesaplama, yüksek hesaplama yükü) ayrı servise çıkarın.

4. "Hızlı Özellik" Hedefine Doğrudan Etki Eden Kararlar

- **Bağımsız deploy hattı:** Her modül kendi CI/CD pipeline'ına sahip olmalı. Yılda 2 sürüm → sürekli teslimat.
- **Event-driven entegrasyon:** Modüller arası senkron çağrı yerine domain event'leri (ör. `TalepOlusturuldu`, `EksperAtandi`). Yeni özellik eklerken mevcut modüllere dokunmadan yeni tüketici eklenebilir.
- **API-first:** Mobil/web fotoğraf yükleme için ayrı, ölçeklenebilir bir medya servisi (object storage + CDN) — transactional DB'ye blob koymayın.
- **Feature flag altyapısı:** Rakiplere hızlı yanıt için özellikleri kademeli açma imkânı.

5. Riskler ve Önlemler

Risk	Önlem
Cobol bilgi kaybı / bus factor	Mevcut ekiple "knowledge capture"; iş kurallarını testlerle belgeleyin
Modül sınırlarının yanlış çizilmesi	Event storming atölyeleri; sınırları pilotta doğrulayın
Veri tutarlılığı (dağıtık)	Başlangıçta tek DB + şema ayrımı; saga'yı yalnızca gerçekten gerekince
100+ geliştirici koordinasyonu	Modül başına otonom takımlar (team topologies), iç platform ekibi
Regülasyon/audit (sigorta sektörü)	Ödeme onayı ve hesaplama için değiştirilemez audit log baştan tasarlanmalı

6. Karar Özeti

Önerim: Modüler monolit ile başlayıp, kanıtlanmış ihtiyaçlar doğrultusunda seçici mikroservis ayrıştırmasına giden kademeli bir yol. "Talep + fotoğraf yükleme" pilotu ile iş değeri erken gösterilmeli; Cobol, Strangler Fig ile kademeli emekliye ayrılmalı. Raporlama, event beslemeli ayrı read-model olarak tasarlanmalı.

Bu yaklaşım, "rakiplerden hızlı özellik" hedefini mimari karmaşıklık borcuna boğulmadan karşılar.